Report on

Hasse Path Finder

hassepathfinder.mahirshah.dev

Prepared by: -

RUMANI VYAS | 2510370910

MAHIR SUJALKUMAR SHAH | 2510370110

CHANDARANA HARSHVI VISHALBHAI | 2510370010

PATEL AVI BALKRUSHNABHAI | 2510370046

SMACT24108

Discrete Mathematics

Faculty of Sciences

B. Tech. in Computer Science & Applied Mathematics

Table of Contents

Sr. No.	Chapter Title	Page No.
1	Introduction	3
2	Relation, Properties & Posets	3
	2.1 What is a Relation?	3
	2.2 Properties of Relations	3
	2.2.1 Reflexive Relation	3
	2.2.2 Anti-Symmetric Relation	4
	2.2.3 Transitive Relation	4
	2.3 Properties of Relations	4
	Hasse Diagram	5
3	Graph Traversal Concepts	6
	3.1 Breath First Search (BFS)	6
	3.1.1 Level-wise Traversal (Layer-by-Layer Exploration)	6
	3.1.2 Use of Queue (FIFO Data Structure)	7
	3.1.3 Avoids Deep Exploration Like DFS	7
	3.1.4 Guarantees Shortest Path in Unweighted Graphs	7
	3.1.5 Maintains Visited Nodes to Prevent Repetition	7
	3.1.6 Suitable for Finding Nearest Solutions	7
	3.1.7 Simple and Predictable Traversal Behaviour	7
	Working of BFS	8
	PSEUDOCODE	9
	JavaScript Code	9
	Example Search using BFS	11
	3.2 Depth First Search (DFS)	
	3.2.1 Level-wise Traversal (Layer-by-Layer Exploration)	11
	3.2.2 Use of Stack (LIFO) or Recursion	11
	3.2.3 Does Not Guarantee Shortest Path	11
	3.2.4 Backtracking Mechanism	11
	3.2.5 More Memory Efficient in Some Cases	12
	3.2.6 Suitable for Exploring All Possible Paths	12
	3.2.7 Simple Implementation and Recursive Nature	12
	PSEUDOCODE	13
	JavaScript Code	13
	Example of Search using DFS	14
4	Using Website	15
5	Real Life Application	17

1. Introduction

Graph theory and discrete mathematics form the backbone of computer science, influencing fields such as artificial intelligence, networking, databases, and algorithm design. Among the various concepts in graph theory, traversal algorithms play a crucial role in exploring structures efficiently. Two of the most fundamental traversal techniques are Breadth First Search (BFS) and Depth First Search (DFS).

This project, titled "Hasse Path Finder," is designed to provide an interactive and visual learning platform that combines graph traversal techniques with partially ordered sets (posets) and Hasse diagrams. Traditional teaching methods often rely heavily on theoretical explanations, which can be difficult for students to grasp. This project bridges that gap by offering a dynamic environment where users can visualize and interact with these concepts.

The tool allows users to input relations, validate them as posets, generate Hasse diagrams, and execute BFS and DFS algorithms step-by-step. This combination of theory and visualization makes learning more intuitive and engaging.

2. Relation, Properties & Posets

2.1 What is a Relation?

In **discrete mathematics**, a **relation** is a way of showing how elements of one set are connected to elements of the same or another set.

Formally: If $\boldsymbol{A}$ is a set, then a relation $\boldsymbol{R}$ on $\boldsymbol{A}$ is a subset of the Cartesian product $\boldsymbol{A} \times \boldsymbol{A}$.

That means: **A relation is just a collection of ordered pairs (a, b) where a and b belong to A.**

Example of a Relation

Let:

$$\boldsymbol{A} = \{\mathbf{1}, \mathbf{2}, \mathbf{3}\}$$

Them:

$$\boldsymbol{A} \times \boldsymbol{A} = \{(\mathbf{1}, \mathbf{1}), (\mathbf{1}, \mathbf{2}), (\mathbf{1}, \mathbf{3}), (\mathbf{2}, \mathbf{1}), (\mathbf{2}, \mathbf{2}), (\mathbf{2}, \mathbf{3}), (\mathbf{3}, \mathbf{1}), (\mathbf{3}, \mathbf{2}), (\mathbf{3}, \mathbf{3})\}$$

Now define:

$$\boldsymbol{R} = \{(\mathbf{1}, \mathbf{1}), (\mathbf{2}, \mathbf{2}), (\mathbf{3}, \mathbf{3}), (\mathbf{1}, \mathbf{2})\}$$

This is a relation because it is a subset of A × A.

2.2 Properties of Relations

For a relation to be useful in structure (like posets), it must satisfy certain properties:

2.2.1 Reflexive Relation: $(a, a) \in R \; \forall \, a \in A$

A relation R on set A is reflexive if every element is related to itself

Example: $A = \{1, 2, 3\}$ $\quad R = \{(1,1), (2,2), (3,3), (1,2)\}$

Relation R is Reflexive as $(1,1), (2,2), (3,3) \in R$
If $R = \{(1,1), (2,2)\}$ then $(3,3) \notin R$ so it would not be Reflexive

2.2.2 Anti-Symmetric Relation: $If\ (a, b) \in R\ \&\ (b, a) \in R \Rightarrow a = b$

If both (a, b) and (b, a) belong to the relation R, then a is equal to b.
Example: $R = \{(1,1), (2,2), (3,3), (1,2)\}$
$(1,2)$ exists but $(2,1)$ does NOT exist, so it is Antisymmetric.
If R = {(1,2), (2,1)} then (1,2) and (2,1) both exist so it is not Antisymmetric.

2.2.3 Transitive Relation: $If\ (a, b) \in R\ and\ (b, c) \in R \Rightarrow (a, c) \in R$

If an element a is related to b, and b is related to c, then a must also be related to c; that is, if (a,b) and (b,c) are in R, then (a,c) is also in R.
Example: $R = \{(1,2), (2,3), (1,3)\}$
$(1,2)$ and $(2,3) \Rightarrow (1,3)$ is present.
If $\mathrm{R} = \{(1,2), (2,3)\}$ If A → B and B → C, then A must directly connect to C.

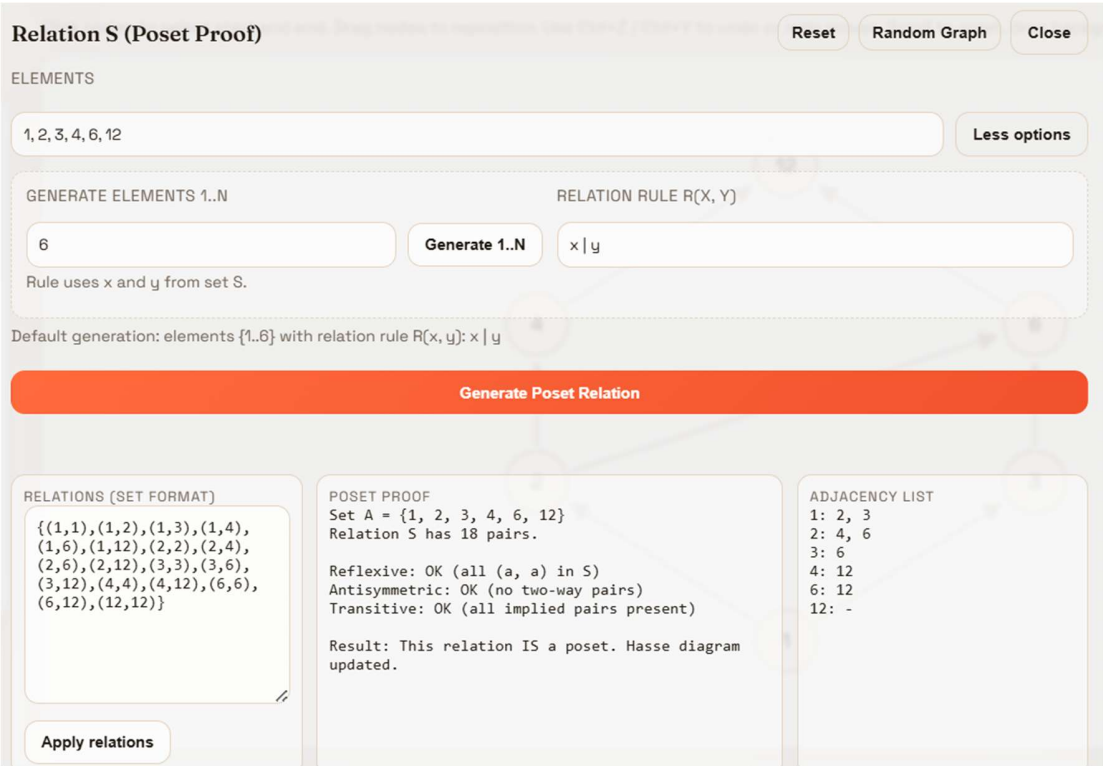
2.3 Properties of Relations

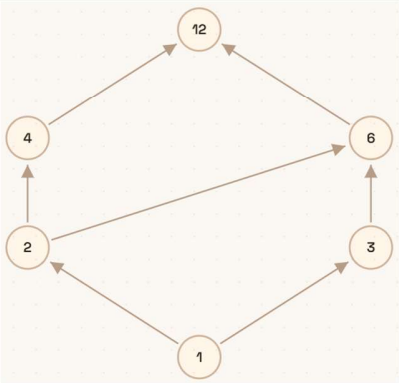
A Partially Ordered Set (Poset) is a set with a relation that is Reflexive, Anti-Symmetric and Transitive. Or in other words A poset is a system where elements follow a **partial order (hierarchy).**
Example:
Let: $A = \{1, 2, 3, 4, 6, 12\}$

$$R = \{(x, y)\ ;\ a \mid b\}$$
$$= \{(1,1), (1,2), (1,3), (1,4), (1,6), (1,12), (2,2), (2,4), (2,6), (2,12), (3,3), (3,6), (3,12), (4,4), (4,12), (6,6), (6,12), (12,12)\}$$

Relation S has 18 pairs.
Reflexive: OK (all (a, a) in S)
Antisymmetric: OK (no two-way pairs)
Transitive: OK (all implied pairs present)
Result: This relation IS a poset. Hasse diagram can be made.

Reference from https://hassepathfinder.mahirshah.dev/





Hasse Diagram

In this poset, the relation used is divisibility, meaning an element a is related to another element b if a divides b. Based on this rule, we determine which elements should be connected directly in the Hasse diagram. These connections are called cover relations, meaning there is no intermediate element between them.

Starting with the smallest element, 1, we observe that it divides all other elements in the set. However, in the Hasse diagram, we do not connect 1 directly to all elements. Instead, we only connect it to those elements which are immediately above it, meaning there is no intermediate divisor between them. Therefore, 1 is connected to 2 and 3, because there is no element between 1 and these two that satisfies the divisibility condition.

Next, we consider the element 2. The number 2 divides 4 and 6. We check if there is any intermediate element between 2 and these numbers. Since there is no such element, we connect 2 to 4 and 2 to 6. However, even though 2 divides 12, we do not connect 2 directly to 12 because there are intermediate elements like 4 and 6. Hence, that connection is removed as a transitive relation.

Similarly, for the element 3, it divides 6 and 12. But again, since 3 divides 6 and 6 divides 12, the direct connection from 3 to 12 is removed. Therefore, we only keep the connection 3 to 6.

Moving further, the element 4 divides 12, and there is no intermediate element between them. Hence, we connect 4 to 12.

Likewise, 6 divides 12, and since no element exists between them, we connect 6 to 12.

Finally, the element 12 does not divide any other element in the set, so it appears at the top of the diagram with no outgoing connections.

Thus, the final structure shows a clear hierarchy: 1 at the bottom, 2 and 3 above it, then 4 and 6, and finally 12 at the top. This structure represents the partial order clearly witout unnecessary edges.

Element	Connected To
1	2, 3
2	4, 6
3	6
4	12
6	12
12	—

3. Graph Traversal Concepts

Graph traversal is the process of visiting all the nodes in a graph in a structured and systematic manner. It ensures that each node is explored without repetition. The two main traversal methods are Breadth First Search (BFS), which explores level by level, and Depth First Search (DFS), which explores deeply before backtracking.

3.1 Breath First Search (BFS)

3.1.1 Level-wise Traversal (Layer-by-Layer Exploration)

Breadth First Search explores nodes in a structured, layer-by-layer manner. It starts from a source node and first visits all nodes that are

directly connected to it (distance 1), then moves to nodes at distance 2, and so on. This systematic expansion ensures that nodes are explored in increasing order of distance, making the traversal predictable and easy to understand.

3.1.2 Use of Queue (FIFO Data Structure)

BFS uses a queue, which follows the First-In-First-Out (FIFO) principle. This means the first node added to the queue is the first one to be processed. The queue ensures that nodes discovered earlier (closer to the source) are explored before nodes discovered later. This is the key mechanism that maintains the level-wise order of traversal.

3.1.3 Avoids Deep Exploration Like DFS

Unlike Depth First Search, BFS does not go deep into one path before exploring others. Instead, it explores all neighboring nodes first before moving deeper. This prevents the algorithm from wasting time on long or unnecessary paths early. As a result, BFS provides a more balanced and controlled exploration of the graph structure.

3.1.4 Guarantees Shortest Path in Unweighted Graphs

One of the most important properties of BFS is that it guarantees the shortest path between the source node and any other node in an unweighted graph. Since BFS explores nodes in increasing distance order, the first time it reaches a node, it has found the minimum number of edges required to reach it. This makes BFS highly useful in pathfinding problems.

3.1.5 Maintains Visited Nodes to Prevent Repetition

BFS keeps track of visited nodes using a separate data structure, usually a set or array. This ensures that each node is processed only once and prevents infinite loops in cyclic graphs. By marking nodes as visited when they are first encountered, BFS avoids redundant work and improves efficiency.

3.1.6 Suitable for Finding Nearest Solutions

BFS is particularly useful when the goal is to find the nearest solution or closest node satisfying a condition. Since it explores all nodes at the current level before moving deeper, it ensures that the first valid solution found is the closest one. This makes BFS ideal for problems like shortest path, minimum steps, and network broadcasting.

3.1.7 Simple and Predictable Traversal Behaviour

BFS has a very predictable pattern of execution because it always follows a level-by-level approach. This makes it easier to trace, debug, and visualize compared to other traversal methods. Its structured behavior is especially useful in teaching and learning environments where understanding the process step-by-step is important.

Working of BFS

1. **Initialization of BFS (Starting the Traversal):**
   The BFS algorithm begins by selecting a starting node, also known as the source node. This node is marked as visited and inserted into a queue. The queue is used to keep track of nodes that need to be explored next. Initialization is important because it sets the base level (level 0) from where the traversal begins.
2. **Enqueuing the Starting Node**
   After selecting the starting node, it is added to the queue. This ensures that it will be processed first. Since BFS uses a First-In-First-Out structure, the starting node will be the first to be removed and explored. This step ensures that traversal begins in a controlled and organized way.
3. **Dequeuing and Processing Nodes**
   The algorithm removes (dequeues) the front node from the queue and processes it. Processing may involve printing the node, checking conditions, or storing it in traversal order. This step ensures that nodes are handled in the correct sequence, maintaining the level-by-level traversal property of BFS.
4. **Exploring All Neighbouring Nodes**
   Once a node is dequeued, BFS explores all of its adjacent (neighboring) nodes. Each neighbor is checked to see whether it has already been visited. This ensures that only new nodes are processed. Exploring neighbors is the key step that expands the traversal to the next level.
5. **Marking Nodes as Visited**
   Before adding any neighbor to the queue, BFS marks it as visited. This prevents the same node from being added multiple times, especially in graphs with cycles. This step is crucial for efficiency and correctness, as it avoids infinite loops and redundant processing.
6. **Repeating the Process Until Queue is Empty**
   All unvisited neighboring nodes are added to the queue. Since the queue follows FIFO order, these nodes will be processed after all nodes currently in the queue. This ensures that nodes at the same level are fully explored before moving to the next level.
7. **Completion of Traversal**
   The algorithm continues the cycle of dequeueing a node, exploring its neighbors, marking them visited, and enqueuing them. This process repeats until the queue becomes empty. An empty queue indicates that all reachable nodes in the graph have been visited.

PSEUDOCODE:

```
BFS(Graph G, start):

    create empty queue Q
    create visited set

    mark start as visited
    enqueue start into Q

    while Q is not empty:
        node = dequeue from Q
        print(node)

        for each neighbor of node:
            if neighbor not visited:
                mark neighbor as visited
                enqueue neighbor into Q
```

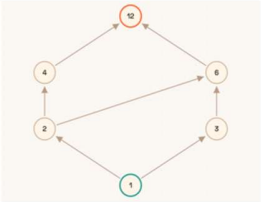
JavaScript Code:

```
function bfs(start, adjacency) {
  const queue = [start];
  const visited = new Set();
  visited.add(start);

  while (queue.length > 0) {
    const node = queue.shift(); // FIFO
    console.log(node);

    for (const neighbor of adjacency[node]) {
      if (!visited.has(neighbor)) {
        visited.add(neighbor);
        queue.push(neighbor);
      }
    }
  }
}
```

Example Search using BFS

Step 1

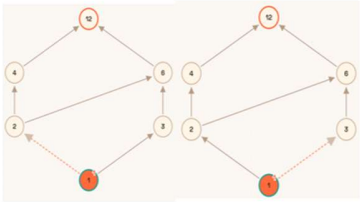
1. Start Node Selected is 1
2. End Node Selected is 12
3. Add First Node to Queue



1					

Step 2

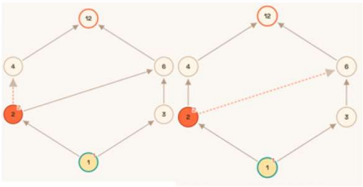
1. Remove First Elements from Queue
2. Add Children of Removed Element to the Queue



	2	3			

Step 3

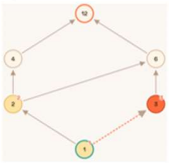
1. Remove First Elements from Queue
2. Add Children of Removed Element to the Queue



		3	4	6	

Step 4

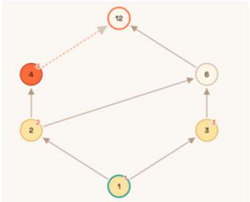
1. Remove First Elements from Queue
2. Add Children of Removed Element to the Queue but already visited



			4	6	

Step 5

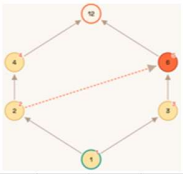
1. Remove First Elements from Queue
2. Add Children of Removed Element to the Queue



				6	12

Step 6

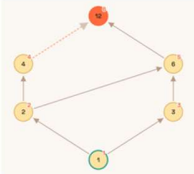
1. Remove First Elements from Queue
2. Add Children of Removed Element to the Queue but **already visited**

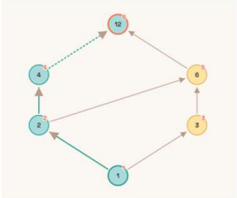


					12

Step 7

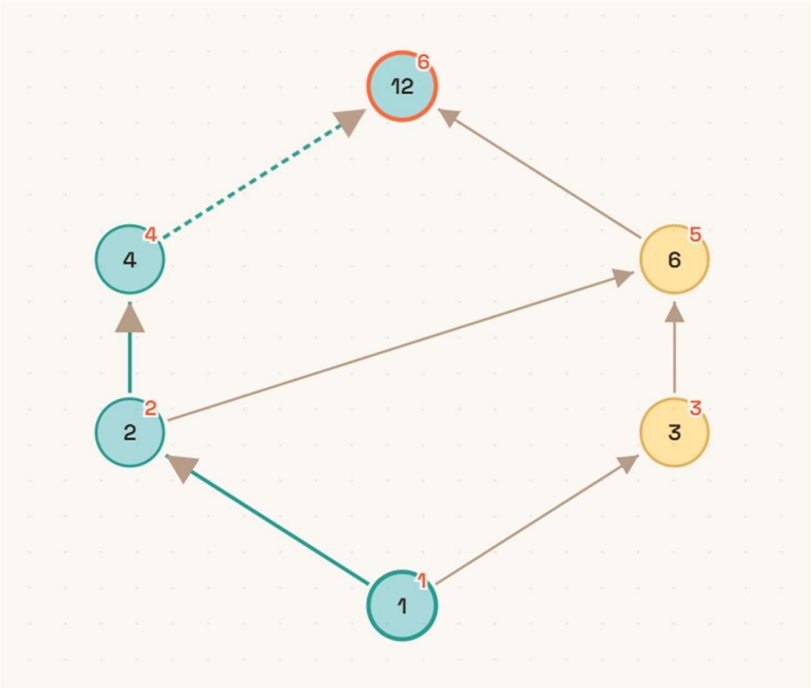
1. Remove First Elements from Queue.
2. No Child, Thus Queue is Empty





Parent Mapping for Shortest Path

1	None
2	1
3	1
4	2
6	2
12	4



3.2 Depth First Search (DFS)

3.2.1 Level-wise Traversal (Layer-by-Layer Exploration)

Depth First Search explores nodes by going as deep as possible along one path before moving to another branch. Starting from a source node, it selects one neighbor and continues moving forward until it reaches a node with no unvisited neighbors. This deep traversal makes DFS different from BFS, which explores level by level instead of going deep first.

3.2.2 Use of Stack (LIFO) or Recursion

DFS uses a stack data structure that follows the Last-In-First-Out (LIFO) principle. This means the most recently added node is processed first, enabling the algorithm to go deeper into the graph. In many implementations, recursion is used instead of an explicit stack, but internally recursion also behaves like a stack, maintaining the order of function calls.

3.2.3 Does Not Guarantee Shortest Path

Unlike BFS, DFS does not guarantee the shortest path in an unweighted graph. This is because DFS may follow a long path before discovering a shorter one. Since it prioritizes depth over distance, it can reach the target node through a longer route even when a shorter path exists, making it unsuitable for shortest path problems.

3.2.4 Backtracking Mechanism

One of the key features of DFS is backtracking. When the algorithm reaches a node that has no unvisited neighbors, it does not stop. Instead, it returns to the previous node and explores alternative paths. This backtracking ensures that all nodes in the graph are eventually visited, even if some paths lead to dead ends.

3.2.5 More Memory Efficient in Some Cases

DFS can be more memory efficient than BFS, especially in graphs where the branching factor is large. Since DFS only needs to store the current path in the stack, it may use less memory compared to BFS, which stores all nodes at the current level. This makes DFS suitable for deep but narrow graphs.

3.2.6 Suitable for Exploring All Possible Paths

DFS is particularly useful when the goal is to explore all possible paths in a graph. It is commonly used in problems such as maze solving, puzzle solving, cycle detection, and topological sorting. Because it goes deep into each branch, it can efficiently explore complete paths before moving to the next one.

3.2.7 Simple Implementation and Recursive Nature

DFS is often easier to implement using recursion, making it simple and elegant in code. The recursive approach closely follows the natural definition of DFS, where the function calls itself for each unvisited neighbor. This simplicity makes DFS a popular choice for many algorithmic problems.

Working of BFS

1. **Start from a Chosen Node (Initialization):**
   The DFS traversal begins by selecting a starting node, also called the source node. This node is marked as visited to ensure it is not processed again. The algorithm then prepares to explore deeper paths from this node, either using a stack or recursion. This step sets the base for the entire depth-first exploration process.
2. **Visit the Node and Mark it as Visited**
   Once the starting node is selected, DFS visits the node and marks it as visited. This step is very important because it prevents revisiting the same node multiple times, especially in graphs with cycles. Marking nodes ensures that the traversal progresses forward without getting stuck in infinite loops.
3. **Move to One of its Unvisited Neighbors**

After visiting the current node, DFS selects one of its unvisited neighboring nodes and moves to it. Unlike BFS, DFS does not explore all neighbors at once. Instead, it picks one path and continues along it. This behavior is what gives DFS its "depth-first" nature.

4. **Repeat the Process (Go Deeper and Deeper)**
   DFS continues this process recursively, moving from one node to another unvisited neighbor. At each step, it goes deeper into the graph, building a path until it reaches a node that has no further unvisited neighbors. This deep exploration helps in discovering complete paths within the graph.
5. **Backtracking When No Unvisited Neighbor is Left**
   When DFS reaches a node where all neighbors have already been visited, it cannot move forward. At this point, it backtracks to the previous node. Backtracking allows the algorithm to revisit earlier nodes and explore any remaining unvisited paths, ensuring full coverage of the graph.
6. **Continue Until All Nodes are Visited**
   The algorithm continues exploring new paths and backtracking whenever necessary until all nodes in the graph have been visited. Once every node is marked as visited, the DFS traversal is complete. This ensures that the entire graph has been explored systematically.

PSEUDOCODE:

```
DFS(Graph G, start):

    create empty stack S
    create visited set

    push start into S

    while S is not empty:
        node = pop from S

        if node not visited:
            mark node as visited
            print(node)

            for each neighbor of node:
                if not visited:
                    push neighbor into S
```

JavaScript Code:

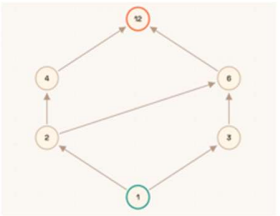
```
function dfs(start, adjacency) {
  const stack = [start];
  const visited = new Set();

  while (stack.length > 0) {
```

```
    const node = stack.pop(); // LIFO

    if (!visited.has(node)) {
      console.log(node);
      visited.add(node);

      for (const neighbor of adjacency[node]) {
        if (!visited.has(neighbor)) {
          stack.push(neighbor);
        }
      }
    }
  }
}
```

Example:

Step 1

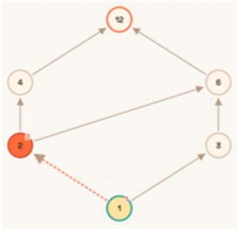
1. Start at node 1.
2. Mark it as visited.
3. From 1, you have two choices → 2 and 3.



Visited	Stack
	1

Step 2

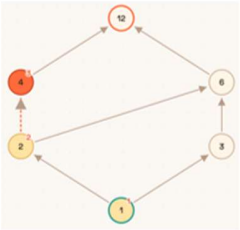
1. Move from 1 → 2.
2. Mark 2 as visited.



Visited	Stack
	4
2	6
1	3

Step 3

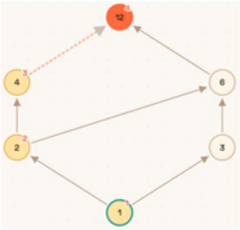
1. Move from 2 → 4.
2. Mark 4 as visited.



Visited	Stack
4	12
2	6
1	3

Step 4

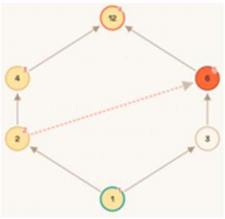
1. Move from 4 → 12.
2. Mark 12 as visited.



Visited	Stack
12	
4	
2	6
1	3

Step 5

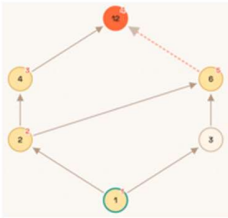
1. Return to previous node → 2
2. Check if any path left



Visited	Stack
6	
12	
4	
2	
1	3

Step 6

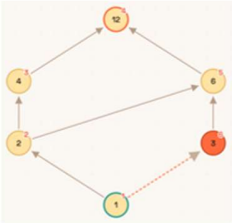
1. Move from 2 → 6.
2. Mark 6 as visited.



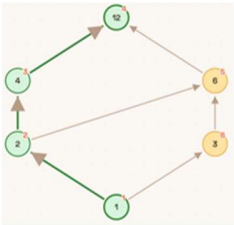
Visited	Stack
6	
12	
4	
2	
1	3

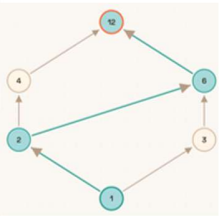
Step 7

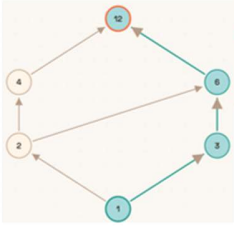
1. Go back to 1
2. Now take the other path → 1 → 3
3. Mark 3 as visited.

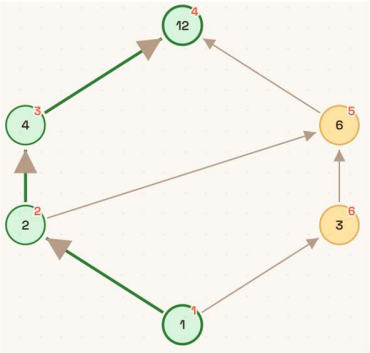


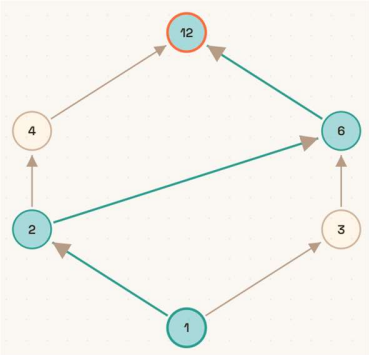
Visited	Stack
3	
6	
12	
4	
2	
1	

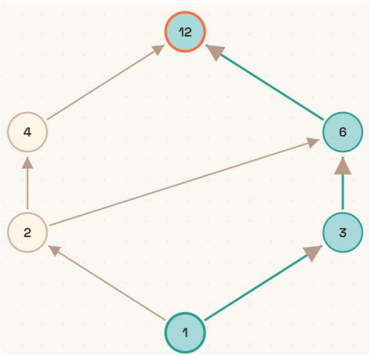












4. Using Website

4.1 Open the Website

First, open the provided website link in a modern web browser such as Google Chrome or Microsoft Edge. Once the page loads, you will see the main interface consisting of a graph display area and a control panel. This interface acts as the working environment where all interactions take place.

4.2 Enter Elements (Set Input)

In the input field labeled "Elements," enter the elements of your set separated by commas. These elements represent the nodes of the graph. For example, you can enter values such as 1, 2, 3, 4, 6, and 12. These values will later be used to construct the relation and generate the Hasse diagram.

4.3 Enter the Relation

Next, define the relation between the elements. This can be done by either entering ordered pairs manually or using predefined rules such as "x divides y (x | y)." The relation determines how the elements are connected and forms the basis for constructing the poset and Hasse diagram.

4.4 Validate the Relation

After entering the relation, click the apply or validate button. The system checks whether the relation satisfies the properties of a poset, namely reflexivity, antisymmetry, and transitivity. If any property fails, the system provides an explanation. If all conditions are satisfied, the relation is confirmed as a valid poset.

4.5 Generate the Hasse Diagram

Once the relation is validated, the system automatically generates the Hasse diagram. The diagram displays nodes arranged in hierarchical levels and shows only the cover relations by removing reflexive and transitive edges. This provides a simplified and clear representation of the partial order.

4.6 Select Start and End Nodes

To perform traversal, select a starting node and an ending node. Use the selection mode options provided in the interface and click on the respective nodes in the diagram. The selected nodes define the traversal path for BFS or DFS algorithms.

4.7 Run BFS or DFS Algorithm

After selecting the nodes, choose either Breadth First Search (BFS) or Depth First Search (DFS). Clicking the corresponding button initiates the traversal. BFS explores nodes level by level, while DFS explores nodes deeply before backtracking. The algorithm execution begins immediately after selection.

4.8 Observe Traversal Animation

During traversal, the system visually highlights the process. The current node is emphasized, visited nodes are marked, and the traversal path is shown step by step. This animation helps users

understand how the algorithm progresses through the graph in real time.

4.9 Use Step Mode for Detailed Understanding

The step mode feature allows users to control the traversal manually. Instead of automatic animation, users can proceed one step at a time using the “Next Step” option. This mode provides detailed explanations of each step, making it especially useful for learning and demonstration purposes.

4.10 View Traversal Log

The system maintains a traversal log that records each step of the algorithm. This includes the nodes visited, the order of traversal, and the path taken. The log helps users review the process and is useful for debugging, analysis, and academic documentation.

4.11 Interact with the Graph

The interface allows users to interact with the graph directly. Nodes can be dragged to adjust layout, and the graph can be zoomed or panned for better visibility. This flexibility helps users analyze complex diagrams more effectively.

4.12 Export the Output

Finally, the generated diagram can be exported in formats such as PNG or SVG. This allows users to include the diagram in reports, presentations, or assignments. The exported image preserves the current visual state of the graph.

Final Workflow Summary

Input Elements → Define Relation → Validate Poset → Generate Hasse Diagram → Select Nodes → Run BFS/DFS → Observe Animation → Export Output

Conclusion

The Hasse Path Finder website provides a complete workflow from defining relations to visualizing traversal algorithms. It integrates theoretical concepts with practical visualization, making it an effective tool for learning and demonstrating discrete mathematics and graph traversal techniques.

5. Real Life Application

1. Shortest Path Finding (BFS)

Breadth First Search is used to find the shortest path in unweighted graphs. It explores nodes level by level, ensuring that the first time a node is reached corresponds to the minimum number of edges required.

2. Network Routing (BFS)

   BFS is used in communication networks to determine efficient routing paths. It helps in finding the shortest route for data packets, reducing delay and improving overall network performance.

3. Maze Solving (DFS)

   Depth First Search is used in maze-solving problems where it explores one path completely before trying others. It is effective in finding a path from start to destination even in complex mazes.

4. Web Crawling (BFS)

   Search engines use BFS-like traversal to crawl web pages. Starting from a root page, it explores all linked pages level by level, ensuring broad and efficient coverage of websites.

5. Cycle Detection (DFS)

   DFS is commonly used to detect cycles in graphs. By tracking visited nodes and recursion stack, it can identify loops, which is useful in deadlock detection and dependency analysis.

6. Social Network Analysis (BFS)

   BFS helps analyze social networks by finding shortest connections between users. It is used in friend suggestions, mutual connections, and determining degrees of separation.

7. Topological Sorting (DFS)

   DFS is used in topological sorting of directed acyclic graphs. It helps determine a valid order of tasks based on dependencies, widely used in scheduling and build systems.

8. Broadcasting in Networks (BFS)

   BFS is used for broadcasting messages in networks. It ensures information spreads evenly from a source node to all reachable nodes, making it useful in communication protocols.

9. Path Existence Checking (DFS)

   DFS is used to check whether a path exists between two nodes. It explores all possible paths deeply until it finds a connection or determines that none exists.

10. Puzzle Solving (DFS)

DFS is widely used in solving puzzles like Sudoku and N-Queens. It explores all possible solutions and backtracks when constraints are violated, ensuring all valid configurations are found.

11. Artificial Intelligence Search (BFS & DFS)

Both BFS and DFS are used in AI search problems. BFS is used for optimal solutions, while DFS is used for exploring deep possibilities in games, decision trees, and problem-solving systems.

12. Garbage Collection (DFS)

DFS is used in memory management to identify unreachable objects. It traverses object references deeply and removes unused objects, helping optimize memory usage

5